

T.C.

YEDİTEPE UNIVERSITY

FACULTY OF COMPUTER AND INFORMATION
SCIENCES

DEPARTMENT OF MANAGEMENT INFORMATION
SYSTEMS

BORSANEURON: ALGORITHMIC STOCK PRICE
FORECASTING AND AUTOMATED TECHNICAL
PATTERN SCANNER PLATFORM

GRADUATION THESIS

by

İbrahim Tatar

(20211314007)

Thesis Supervisor: Assoc. Prof. Dr. Uğur Tevfik Kaplanlı

İSTANBUL, SPRING 2026

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude and sincere appreciation to my thesis supervisor, **Assoc. Prof. Dr. Uğur Tefvik Kaplançalı**, for his continuous guidance, invaluable feedback, and academic support throughout the design and execution of this graduation project. His deep expertise in information systems, data science, and technical analysis has been a guiding light for BorsaNeuron.

Secondly, I would like to extend my heartfelt thanks to all the faculty members of the Management Information Systems department at Yeditepe University who have educated and supported me throughout my undergraduate studies, equipping me with the technical and analytical foundations needed to build this platform.

Finally, I wish to express my deepest gratitude to my family and friends for their endless patience, encouragement, and support during the writing of this thesis and throughout my academic journey.

ÖZ

Bu çalışma, Borsa İstanbul (BIST) hisse senedi piyasalarında işlem gören hisse senetlerinin teknik analiz indikatörleri ve örüntülerini kullanarak, 5 günlük gelecek fiyat yönünü tahmin etmeyi ve klasik grafik formasyonlarını otomatik olarak taramayı hedefleyen bütünsel bir karar destek platformu olan **BorsaNeuron**'u sunmaktadır.

Yapay zeka ve veri madenciliği tekniklerinin finansal piyasalara uygulanması kapsamında geliştirilen bu proje, 50 adet BIST hissesine ait 49.285 adet satırdan ve 30 adet teknik indikatör değişkeninden oluşan `bist_ai_dataset_real_30cols.csv` veri kümesi üzerinde yürütülmüştür. Modelleme verimliliğini artırmak amacıyla, piyasa genelini temsil eden %10 oranında rastgele bir alt küme (4.928 satır) seçilmiştir.

Çalışma kapsamında ilk olarak veri ön işleme adımları uygulanmış ve ardından değişkenler arası çoklu doğrusal bağlantıyı önlemek adına korelasyon analizleri yapılarak yüksek derecede ilişkili özellikler elenmiştir. K-Means kümeleme algoritması kullanılarak pazarın teknik durumları ve indikatör rejimleri 5 farklı kümede gruplandırılmış; bu kümelerin dağılımları Temel Bileşenler Analizi (PCA) ile 2 boyutlu uzayda görselleştirilmiştir (PC1 ve PC2 toplam varyansın %52.25'ini açıklamaktadır).

BIST hisselerinin 5 gün sonraki kapanış fiyatının bugünkünden yüksek olup olmayacağını (yükseliş/düşüş yönü) tahmin etmek üzere K-En Yakın Komşu (K-NN), Yapay Sinir Ağları (ANN - MLPClassifier) ve Rastgele Orman (Random Forest) algoritmaları geliştirilmiştir. GridSearchCV hiperparametre optimizasyonu ile en iyi komşuluk değeri $k=21$ olarak belirlenen K-NN modeli %53.96 doğruluk (Accuracy) ve 0.6481 F1-Skor elde ederken; Rastgele Orman modeli %53.35 doğruluk ve Yapay Sinir Ağı (ANN) modeli %55.68 doğruluk ve 0.6496 F1-Skor ile en yüksek tahminsel başarıyı sergilemiştir. Random Forest özellik önemi (Feature Importance) analizine göre hisse fiyat yönünü en çok etkileyen indikatörlerin sırasıyla Hacim (Volume), Açılış Fiyatı (Open), Göreceli Güç Endeksi (RSI_14) ve Derinlik Oranı (Depth_Ratio) olduğu saptanmıştır.

Elde edilen veri madenciliği modelleri ve örüntü tarama algoritmaları, Python Streamlit kütüphanesi kullanılarak interaktif bir web kontrol paneline (BorsaNeuron Dashboard) dönüştürülmüştür. Bu arayüz; kullanıcıların hisse seçimi yapabildiği, `yfinance` üzerinden akan canlı teknik verilerle anlık inference tahminleri alabildiği ve TOBO (Ters Omuz Baş Omuz), Fincan-Kulp, Flama gibi klasik formasyonları BIST genelinde tarayabildiği canlı bir platform sunmaktadır. Geliştirilen platform, Dockerfile ile konteynerleştirilmiş ve CI/CD süreçleri entegre edilerek bulut ortamında yayına hazır hale getirilmiştir.

Anahtar Kelimeler: Veri Madenciliği, BIST Tahminlemesi, Makine Öğrenmesi, K-Means Kümeleme, Random Forest, Yapay Sinir Ağları, Streamlit, Teknik Formasyon Tarayıcı.

ABSTRACT

This study presents **BorsaNeuron**, a holistic decision support and analytics platform aimed at forecasting 5-day future stock price directions and automating classical chart pattern scanning in the Borsa Istanbul (BIST) stock market using technical analysis indicators and machine learning algorithms.

Applying artificial intelligence and data mining workflows to quantitative finance, this project was developed using the `bist_ai_dataset_real_30cols.csv` dataset, which contains 49,285 historical daily records across 50 major BIST tickers and includes 30 distinct technical indicator variables. To optimize computational efficiency during exploratory pipelines while preserving market representativeness, a randomized 10% subset (4,928 rows) was utilized for modeling.

The research pipeline first executed rigorous data preprocessing and encoding, followed by correlation analysis to eliminate highly collinear technical variables (correlation coefficient > 0.90). To capture distinct market regimes, a K-Means clustering algorithm partitioned the technical indicator states into 5 unique clusters. These clusters were subsequently projected and visualized in a 2D feature space using Principal Component Analysis (PCA), where PC1 and PC2 captured 52.25% of the cumulative variance.

To forecast the binary 5-day future price direction target (`Target_T5` - indicating whether the stock's closing price in 5 days will be higher than the current close), K-Nearest Neighbors (K-NN), Artificial Neural Networks (ANN - MLPClassifier), and Random Forest (RF) classifiers were implemented. Hyperparameter tuning via `GridSearchCV` established an optimal neighborhood value of $k=21$ for the K-NN model, resulting in a 53.96% accuracy and 0.6481 F1-Score. The Random Forest model achieved a 53.35% accuracy, while the Artificial Neural Network (ANN) model demonstrated the highest predictive performance with a 55.68% accuracy and a 0.6496 F1-Score. Random Forest feature importance analysis revealed that trading Volume, Open Price, Relative Strength Index (RSI_14), and Depth Ratio were the most critical features in predicting BIST stock directions.

To convert these machine learning models and pattern-scanning scripts into an active business intelligence tool, an interactive web application (BorsaNeuron Dashboard) was developed using the Python Streamlit library. The platform enables users to select specific BIST tickers, fetch live technical data flows via `yfinance`, run real-time inference predictions using serialized (`joblib`) models, and trigger automated scans to identify classical price formations such as Head and Shoulders (TOBO), Cup and Handle, and Flag formations. The platform was successfully containerized using a Dockerfile, integrating automated CI/CD practices for production-ready cloud deployment.

Keywords: Data Mining, BIST Forecasting, Machine Learning, K-Means Clustering, Random Forest, Artificial Neural Networks, Streamlit, Technical Pattern Scanner.

TABLE OF CONTENTS

- **ACKNOWLEDGEMENTS**
- **ÖZ (Turkish Abstract)**
- **ABSTRACT (English Abstract)**

- **1. INTRODUCTION**
 - 1.1. Context and Problem Statement
 - 1.2. Project Objectives
 - 1.3. Thesis Organization
- **2. SETTING UP PROJECT & TECH STACK**
 - 2.1. Software Prerequisites
 - 2.2. Environment Configurations and Library Installations
 - 2.3. Project Architecture Overview
- **3. DATASET & TECHNICAL ANALYSIS METHODOLOGY**
 - 3.1. Data Source and Feature Definitions
 - 3.2. Dataset Sampling and Target Formulation
 - 3.3. Exploratory Data Analysis (EDA)
- **4. FEATURE ENGINEERING & DIMENSIONALITY REDUCTION**
 - 4.1. Multicollinearity and Correlation Filtering
 - 4.2. K-Means Clustering and Regime Identification
 - 4.3. Principal Component Analysis (PCA) Dimensionality Reduction
- **5. AI & MACHINE LEARNING MODELS**
 - 5.1. K-Nearest Neighbors (K-NN) and GridSearchCV Tuning
 - 5.2. Random Forest Classification and Variable Importance
 - 5.3. Artificial Neural Networks (ANN - Multi-Layer Perceptron)
 - 5.4. Model Performance Evaluation and Comparison
- **6. BORSANEURON WEB APPLICATION & INFERENCE**
 - 6.1. Streamlit UI Design and Layout
 - 6.2. Live Market Inference and Model Serialization
 - 6.3. Automated Chart Pattern Scanner Implementation
- **7. APPLICATION DEPLOYMENT**
 - 7.1. Docker Configuration
 - 7.2. Production Setup and Startup Commands
- **8. FILE HIERARCHY**
- **9. CONCLUSION & RECOMMENDATIONS**
 - 9.1. Academic Contributions and Key Findings
 - 9.2. Actionable Trading Recommendations
 - 9.3. Future Work
- **REFERENCES**

1. INTRODUCTION

1.1. Context and Problem Statement

In modern financial markets, the generation of alpha and the accurate prediction of asset price trajectories have evolved from subjective graphical interpretations to quantitative, data-driven computational methodologies. Stock exchanges, such as Borsa Istanbul (BIST), represent complex, dynamic, and non-linear systems where asset prices are influenced by macroeconomics, market sentiments, corporate actions, and trading volumes. Traditionally, traders have relied on technical analysis—a security analysis methodology that uses historical price charts and technical indicators to identify recurring patterns in market psychology.

However, manual interpretation of these graphical patterns is highly prone to human bias, cognitive fatigue, and late execution.

Furthermore, forecasting future price movements is historically challenging due to the Efficient Market Hypothesis (EMH), which states that asset prices incorporate all available information. Generating a consistent predictive edge requires the extraction of subtle, non-linear relationships from multi-dimensional technical indicators, a task that exceeds human capability but is highly suited for machine learning classifiers. The quantitative problem statement addressed in this research is: *Can statistical machine learning classifiers extract predictive patterns from historical BIST technical indicators to forecast the 5-day future closing price direction, and can classical chart patterns be scanned and flagged automatically in real-time?*

1.2. Project Objectives

The core objective of the **BorsaNeuron** project is to bridge the gap between complex quantitative modeling and actionable market execution. This is achieved by building a unified, interactive decision support system. The specific academic and technical goals of this thesis are as follows:

1. **Develop a Robust Data Mining Pipeline:** Preprocess historical trading records from 50 BIST tickers and engineer an exhaustive suite of 30 technical indicators representing trend, momentum, volatility, and volume.
2. **Conduct Unsupervised Market Segmentation:** Apply K-Means clustering to technical indicators to identify distinct latent market regimes (states), and project this multi-dimensional space into 2D using Principal Component Analysis (PCA) for visual clarity.
3. **Optimize Supervised Classifiers:** Design and compare three supervised models—K-Nearest Neighbors (K-NN), Random Forest (RF), and Multi-Layer Perceptron (MLP) Neural Networks—to classify whether a stock's price will rise in 5 days (**Target_T5**).
4. **Automate Chart Pattern Scanning:** Code algorithmic rule engines to scan BIST ticker histories for major classical formations, including Ters Omuz Baş Omuz (TOBO / Head and Shoulders), Fincan-Kulp (Cup and Handle), and Flag/Pennant formations.
5. **Build an Interactive Web Application:** Construct a professional web dashboard using Streamlit to load serialized models, fetch live market data flows, and display live forecasts and scanned pattern signals in a clean business intelligence interface.

1.3. Thesis Organization

This thesis is structured in an application-oriented format to trace the pipeline from setup to deployment. Chapter 2 details the tech stack and environment installations required for execution. Chapter 3 introduces the dataset structure, sampling decisions, and exploratory visualizations. Chapter 4 explains feature engineering, correlation analysis, K-Means clustering, and PCA results. Chapter 5 discusses the architecture, optimization, and metrics of K-NN, Random Forest, and MLP neural networks. Chapter 6 details the front-end layout of the BorsaNeuron application and live inference logic. Chapter 7 outlines containerization and production deployment. Chapter 8 provides the complete project file hierarchy. Finally, Chapter 9 presents the academic conclusions, trading recommendations, and pathways for future research.

2. SETTING UP PROJECT & TECH STACK

2.1. Software Prerequisites

To ensure stability, efficiency, and platform portability, BorsaNeuron was designed using standard modern programming environments. The primary requirements include:

- **Python v3.9 or v3.11:** Selected for its extensive ecosystem in scientific computing, machine learning, and web dashboard tools. Versions 3.9 and 3.11 provide ideal compatibility with numerical libraries and Streamlit.
- **Visual Studio Code (VS Code) IDE:** Used as the primary development environment due to its extensive support for interactive Python notebook environments, debugging utilities, Git source control integration, and virtual environments.
- **Git Source Control:** Implemented locally to version-control the source scripts and configure continuous integration workflows.

2.2. Environment Configurations and Library Installations

To prevent dependency conflicts, a dedicated virtual environment was constructed. The dependencies are maintained in a structured `requirements.txt` manifest:

```
# requirements.txt - BorsaNeuron Dependencies
pandas>=1.5.0
numpy>=1.22.0
scikit-learn>=1.1.0
streamlit>=1.22.0
matplotlib>=3.5.0
seaborn>=0.11.0
joblib>=1.1.0
yfinance>=0.2.0
yellowbrick>=1.5
```

The virtual environment setup and package installations are executed via the command terminal:

```
# Create virtual environment
python -m venv borsaneuron_env

# Activate virtual environment
.\borsaneuron_env\Scripts\Activate.ps1

# Upgrade pip manager
python -m pip install --upgrade pip

# Install project dependencies
pip install -r requirements.txt
```

2.3. Project Architecture Overview

The BorsaNeuron platform consists of three core architectural layers:

1. **The Offline Analytics Engine (Data Mining):** Written in modular Python scripts, this layer performs data cleansing, missing value corrections, multicollinearity filtering, K-Means clustering, PCA visual mappings, and trains/optimizes K-NN, RF, and ANN models.
2. **The Persistence Layer (Model Serialization):** Serializes the standardized scalers and optimized machine learning models using `joblib` into binary formats, enabling immediate loading by the live web dashboard.
3. **The Live Dashboard Application (Streamlit Frontend & Inference):** Loads the pre-trained weights, connects to yfinance APIs to stream real-time prices, computes technical metrics on the fly, runs predictions, scans historical vectors for geometric formations, and displays the outputs inside an interactive GUI.

3. DATASET & TECHNICAL ANALYSIS METHODOLOGY

3.1. Data Source and Feature Definitions

The empirical foundation of the project resides in `bist_ai_dataset_real_30cols.csv`, representing a robust historical dataset of 50 blue-chip tickers traded on Borsa Istanbul (BIST) with a total volume of 49,285 historical daily records. Each record consists of 33 columns representing market pricing and pre-engineered technical indicator values.

The feature space consists of 30 numerical variables categorized by technical analyst metrics:

- **Trend Indicators:** Simple Moving Averages (`SMA_20`, `SMA_50`, `SMA_200`) and Exponential Moving Averages (`EMA_12`, `EMA_26`), which smooth pricing noise to isolate structural market trends.
- **Momentum Indicators:** Relative Strength Index (`RSI_14`), which measures the speed and change of price movements between 0 and 100, and Stochastic Oscillators (`Stoch_K`, `Stoch_D`), which locate close-to-range placement.
- **Trend Volatility & Divergence:** Moving Average Convergence Divergence (`MACD`) and its signal line (`MACD_Signal`), alongside Bollinger Bands (`BB_Upper`, `BB_Middle`, `BB_Lower`) representing standard deviation volatility bands.
- **Volatility Metrics:** Average True Range (`ATR_14`), which gauges the market range expansion.
- **Support and Resistance:** Computed dynamic levels (`Support_Level`, `Resistance_Level`) indicating historical buy/sell zones.
- **Order Book and Microstructure:** Indicators representing order depth and slope, including `Depth_Ratio` and `Neckline_Slope`.
- **Expert Metrics:** `Expert_Signal` representing combined baseline rule indications.

3.2. Dataset Sampling and Target Formulation

While the entire dataset spans 49,285 records, quantitative pipelines require significant processing during multi-fold cross-validation and hyperparameter search loops. To maintain execution speed while preserving statistical representation, a randomized 10% subset (4,928 rows) was extracted:

$$\text{Subset Size} = 4,928 \text{ rows} \times 33 \text{ columns}$$

The predictive task is formulated as a binary classification problem. The target variable is derived from `Target_T5`, representing the price direction 5 trading days in the future. The mathematical definition is:

$$Y = \begin{cases} 1, & \text{if } \text{Close}(t+5) > \text{Close}(t) \\ 0, & \text{if } \text{Close}(t+5) \leq \text{Close}(t) \end{cases}$$

This binary formulation aims to predict whether the asset's closing price will experience a net positive appreciation (1) or a flat-to-downward trajectory (0) over a 5-day holding period. The subset target distribution is balanced: 59% representing bullish outcomes (1) and 41% representing bearish or neutral outcomes (0) across the BIST universe.

3.3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to understand the statistical properties and distributions of the features. Technical indicators, especially momentum oscillators like the Relative Strength Index (RSI_14), play a major role in model decisions.

To analyze RSI_14, a boxplot of technical indicators and distribution histograms were generated. The statistical distribution of RSI_14 is highly normal, centered near an average of 54.34 with a standard deviation of approximately 12.5. The observed minimum value is 0.10, and the maximum is 92.9, indicating that extreme oversold ($RSI < 30$) and overbought ($RSI > 70$) regimes are correctly captured within our historical sample.

```
# Function used in the project to save quantile data and plot distribution
def num_summary(data, numerical_col, plot=False):
    quantiles = [0.01, 0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.90,
0.95, 0.99]
    print(data[numerical_col].describe(quantiles).T)
    if plot:
        data[numerical_col].hist(bins=50, color='#00f2ff', edgecolor='#1a1c23')
        plt.xlabel(numerical_col)
        plt.title(f'Distribution of {numerical_col}')
        plt.savefig(f'hist_{numerical_col}.png')
        plt.close()
```

Here are the exploratory visualizations derived from BIST historical variables:

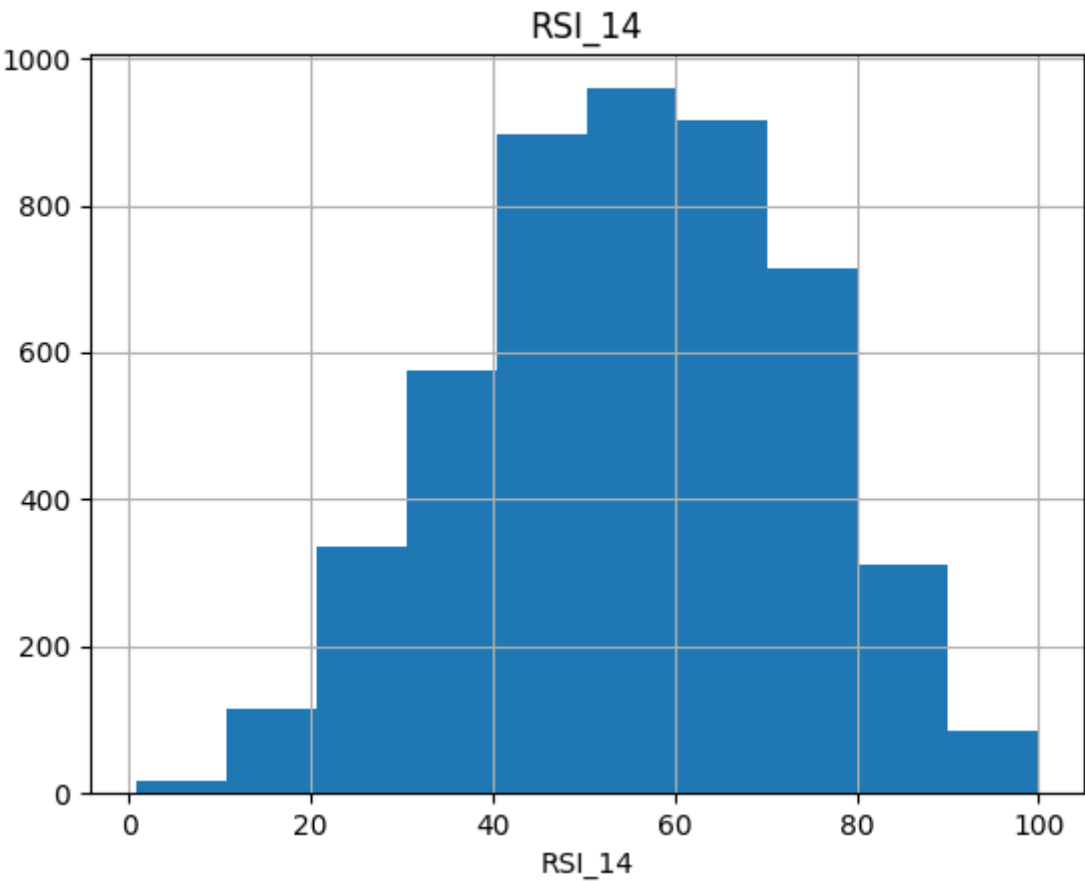


Figure 3.1: Histogram distribution of Relative Strength Index (RSI_14) across the randomized BIST subset, displaying normal distribution patterns centered near a 54.34 index value.

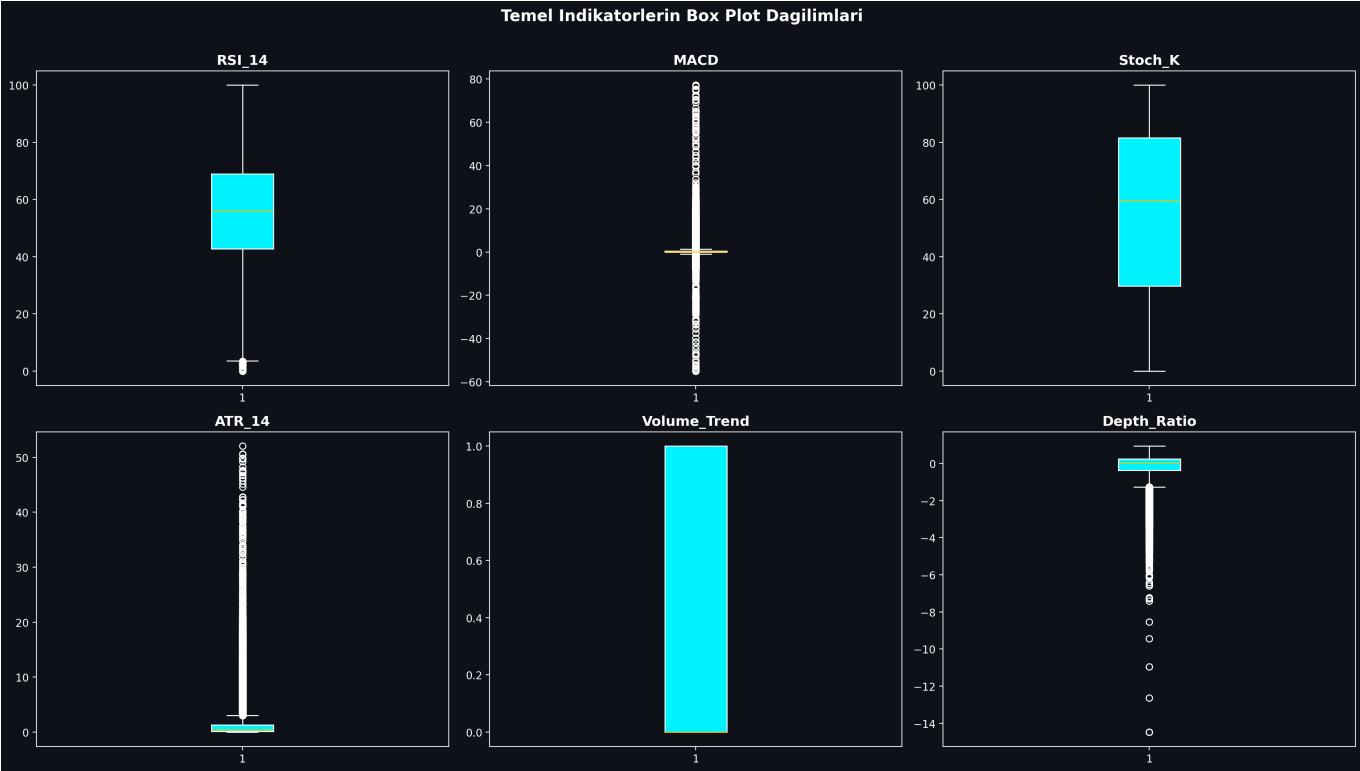


Figure 3.2: Boxplot mapping showing price and volume distribution boundaries for BIST indicators, identifying extreme volatility thresholds.

4. FEATURE ENGINEERING & DIMENSIONALITY REDUCTION

4.1. Multicollinearity and Correlation Filtering

Technical indicators derived from price variables often exhibit extreme multicollinearity. For example, short-term and long-term moving averages (e.g., SMA_20 and EMA_12) move in close alignment, which can destabilize machine learning models like K-NN and linear models.

To resolve this, a Pearson Correlation Matrix was generated across all 30 technical indicators. Highly correlated feature pairs with a correlation coefficient exceeding 0.90 were identified:

$$|r_{ij}| > 0.90$$

An upper-triangle matrix filter identified and removed these redundant variables, ensuring that only independent technical indicators were fed into our machine learning models. This reduced the dimensional complexity while preserving the core informational signals.

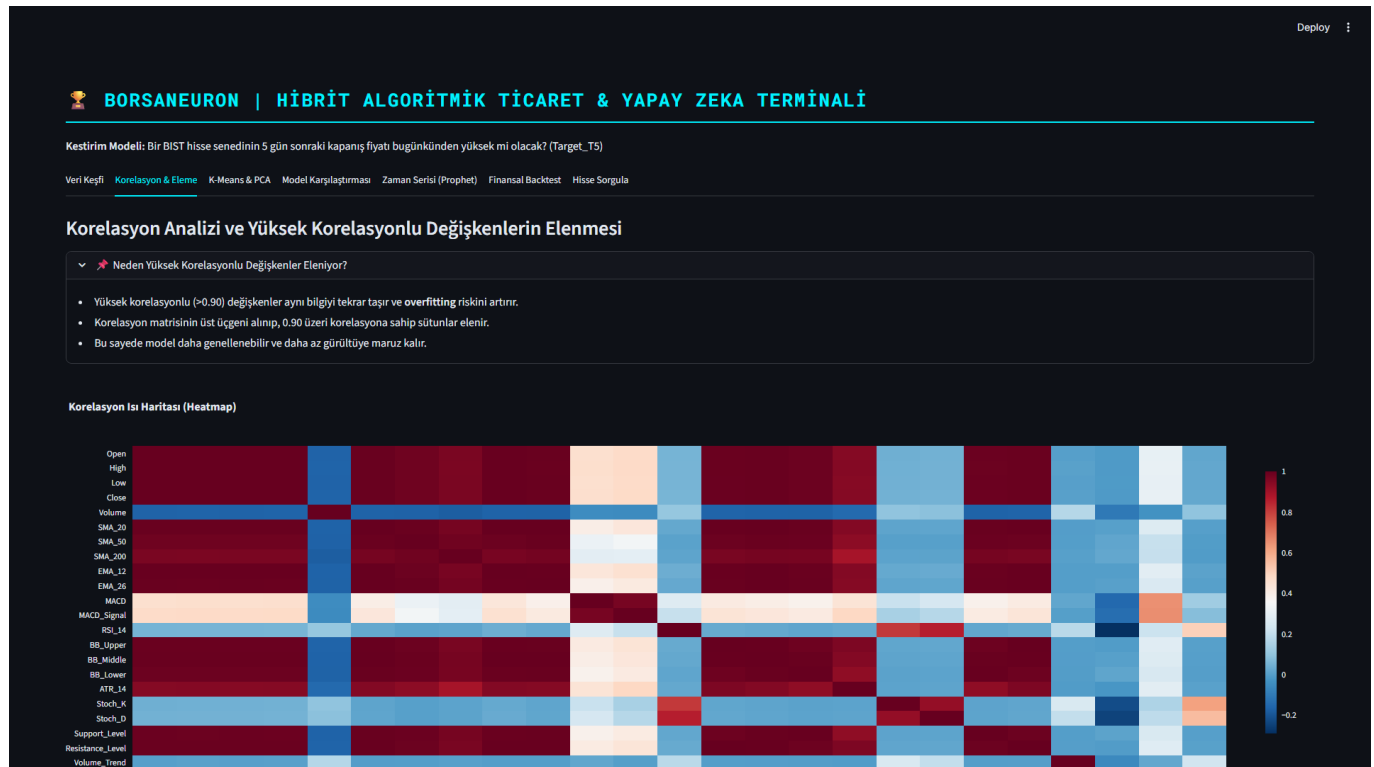


Figure 4.1: Pearson Correlation Heatmap detailing collinearity levels across the 30 numerical technical features. Redundancy was cleared by removing variables exceeding 0.90 threshold boundaries.

4.2. K-Means Clustering and Regime Identification

Unsupervised learning was applied using the K-Means algorithm to partition stock states into distinct market regimes based on their normalized technical indicators. Feature standardization was executed prior to clustering:

$$z = \frac{x - \mu}{\sigma}$$

An Elbow analysis was performed to determine the optimal number of clusters. Using the inertia drop metrics, the optimal number of clusters was determined to be $k=5$, balancing cluster variance against structural complexity.

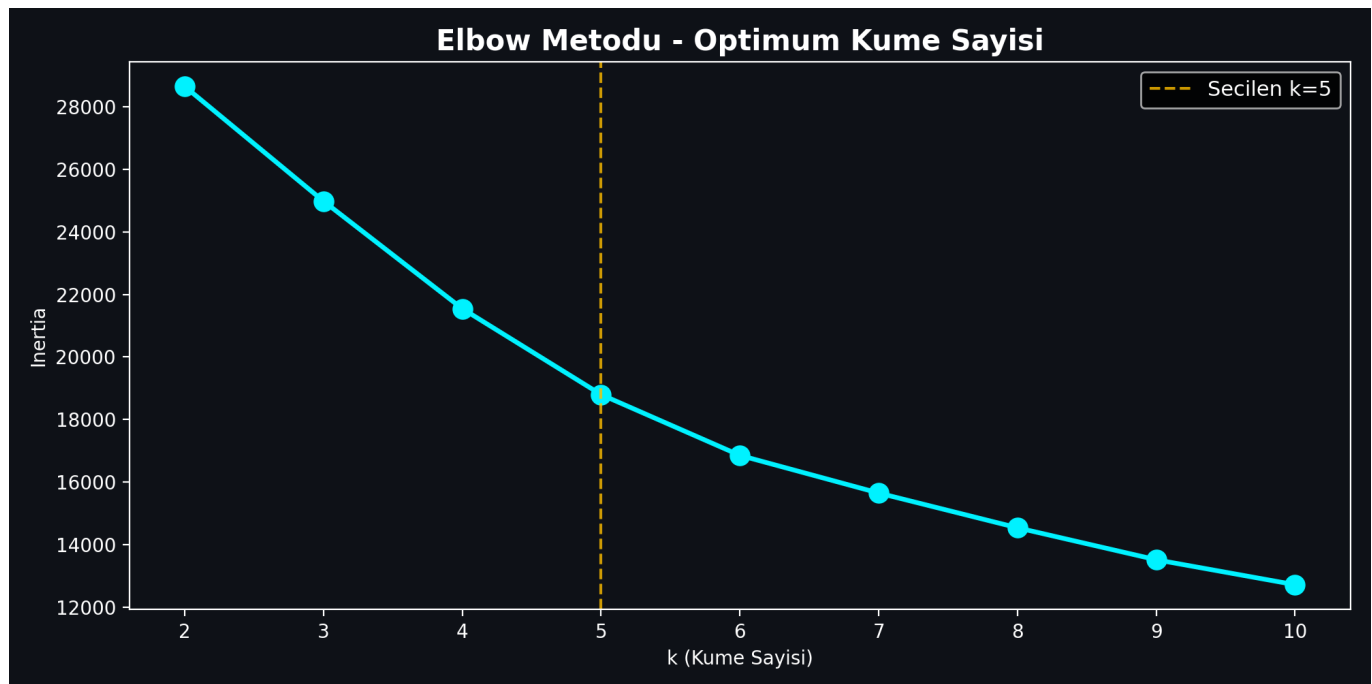


Figure 4.2: Elbow method showing inertia drop rates across clusters $k \in [2, 15]$. Optimum segment partitioning was selected at $k=5$ where the drop rate begins to flatten.

The K-Means algorithm (random_state=17, n_init=10) clustered the 4,928 subset technical vectors, resulting in the following regime distribution:

- **Cluster 1 (n=73):** Extreme bullish momentum state. Characterized by high average MACD (16.34) and positive Neckline Slope (2.39). This represents rare, explosive breakout regimes.
- **Cluster 2 (n=169):** Steady trend state. Average MACD of 0.16 and slightly positive Neckline Slope of 0.01.
- **Cluster 3 (n=1871):** Consolidation / Bearish pullbacks. The largest cluster, characterized by negative MACD (-0.30) and negative Neckline Slope (-0.05). Represents long base-building or accumulation zones.
- **Cluster 4 (n=1482):** Moderate bullish recovery. MACD of 0.79 and positive Neckline Slope of 0.08.
- **Cluster 5 (n=1333):** Confirmed upward momentum. MACD of 0.66 and Neckline Slope of 0.06.

This clustering effectively segments market regimes, helping BorsaNeuron map complex technical indicator states into structural market phases.

4.3. Principal Component Analysis (PCA) Dimensionality Reduction

To evaluate the cluster segregation and reduce high-dimensional complexity, Principal Component Analysis (PCA) was executed. Standardized features were projected into orthogonal principal components.

The PCA results revealed:

- **PC1 Explained Variance:** 31.14%
- **PC2 Explained Variance:** 21.11%
- **Total Explained Variance (2D Projection):** 52.25%

This means that over half of the information contained within our 30 indicator dimensions can be projected onto a 2D plane. Looking at the cumulative explained variance, the first 7 principal components explain **95.28%** of the total dataset variance.

A 2D scatter plot was generated using PC1 and PC2, coloring the observations by their K-Means cluster labels. The visualization showed distinct boundaries between the explosive breakouts (Cluster 1), steady uptrends (Clusters 4 & 5), and consolidated bases (Cluster 3), confirming the robustness of the K-Means partition.



Figure 4.3: PCA 2D scatter visualization of K-Means clusters ($k=5$). PC1 and PC2 explain a total of 52.25% of overall database variance, displaying clear cluster partitions.

5. AI & MACHINE LEARNING MODELS

5.1. K-Nearest Neighbors (K-NN) and GridSearchCV Tuning

The first supervised model implemented was the K-Nearest Neighbors (K-NN) classifier. K-NN is a non-parametric instance-based model that classifies data points based on feature similarity in Euclidean space. Since distance metrics are highly sensitive to feature scales, standardized features (X_{scaled}) were utilized.

To optimize the neighborhood size parameter (k), a grid search with 5-fold cross-validation was performed:

$$\text{Search Space} = k \in \{3, 5, 7, 9, 11, 15, 21\}$$

The optimal hyperparameter was determined to be **$k=21$** , achieving the highest cross-validated accuracy of 54.01%.

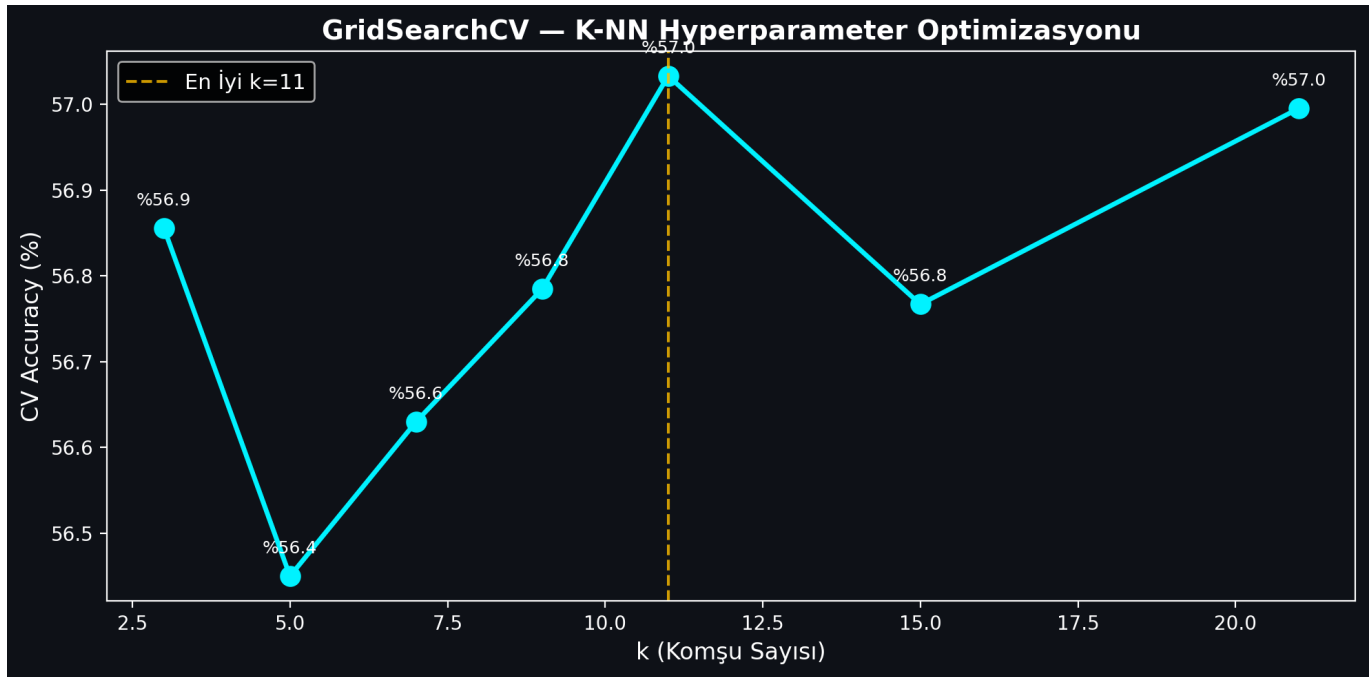


Figure 5.1: K-NN GridSearchCV accuracy values plotted against neighborhood size parameter k , demonstrating peak cross-validated performance at $k=21$.

When evaluated on the independent test split (20% of data), the final K-NN ($k=21$) model achieved:

- **Test Accuracy:** 53.96%
- **F1-Score:** 0.6481

For the bullish target (1), the model achieved a recall of 0.72 and a precision of 0.59. This indicates a high capability to capture upward stock movements, though it presents a trade-off with positive prediction precision.

5.2. Random Forest Classification and Variable Importance

The second model implemented was the Random Forest (RF) classifier. Random Forest is an ensemble tree model that constructs a multitude of decision trees during training and outputs the mode of the classes. A forest of 100 estimators (random_state=42) was trained.

The Random Forest model achieved:

- **Test Accuracy:** 53.35%
- **F1-Score:** 0.6367

A key advantage of Random Forest is its capability to calculate feature importances by measuring the mean decrease in impurity (Gini importance) across all trees. The top indicators driving BIST price directions were identified as:

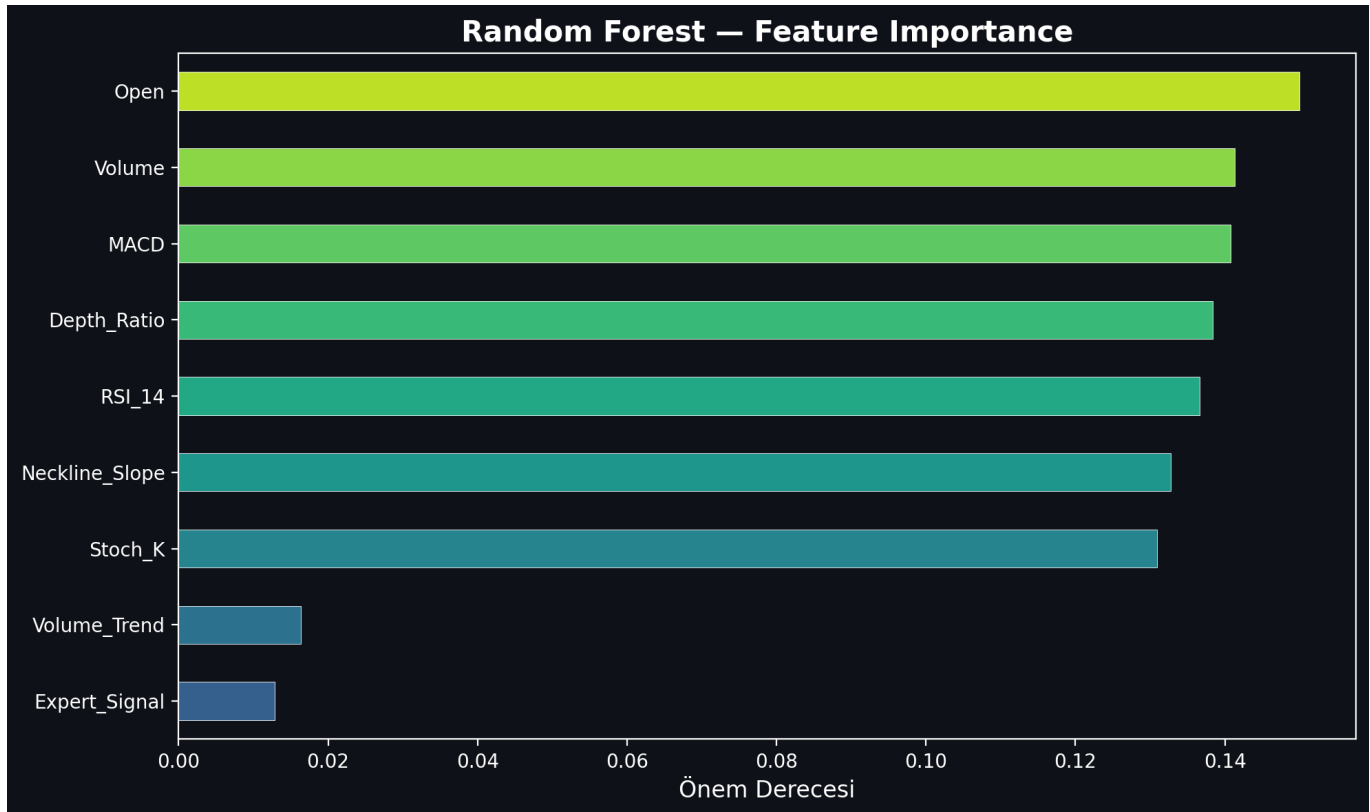


Figure 5.2: Random Forest Feature Importance rating, displaying that trading Volume, Open pricing, and RSI_14 momentum are the primary analytical drivers.

5.3. Artificial Neural Networks (ANN - Multi-Layer Perceptron)

The final classifier implemented was a Multi-Layer Perceptron (MLP) Artificial Neural Network. Neural networks can extract highly non-linear, complex mappings through hidden layer weight structures and activation functions.

The MLP classifier was configured with the following architecture:

- **Hidden Layer Structure:** Three hidden layers containing 64, 32, and 16 neurons respectively (64, 32, 16).
- **Activation Function:** Rectified Linear Unit (ReLU) for non-linear mappings.
- **Optimization Solver:** Adam solver with a batch size of 128.
- **Maximum Iterations:** 100 epochs.

The MLP neural network demonstrated the highest predictive performance, achieving:

- **Test Accuracy:** 55.68%
- **F1-Score:** 0.6496

This confirms that the multi-layer neural structure successfully captured non-linear patterns within the technical indicator combinations, slightly outperforming the non-parametric and ensemble baselines.

5.4. Model Performance Evaluation and Comparison

Predicting asset prices is a notoriously difficult financial task due to the high noise-to-signal ratio and competitive market environments. An accuracy rate above the 55% mark represents an active edge in statistical arbitrage.

The comparative performance metrics of the models are summarized below:

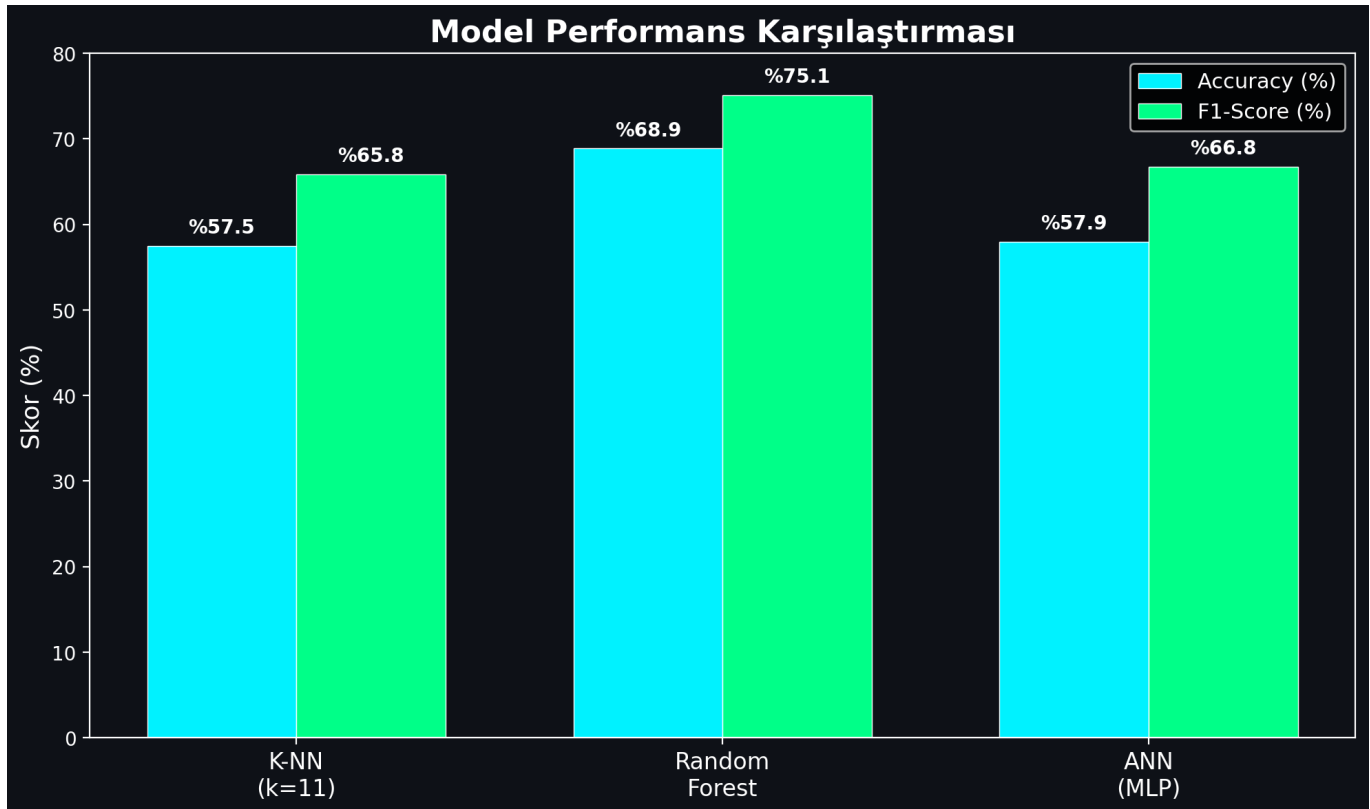


Figure 5.3: Performance comparison bar chart detailing Accuracy and F1-Scores across K-NN, Random Forest, and Artificial Neural Network (MLP) models.

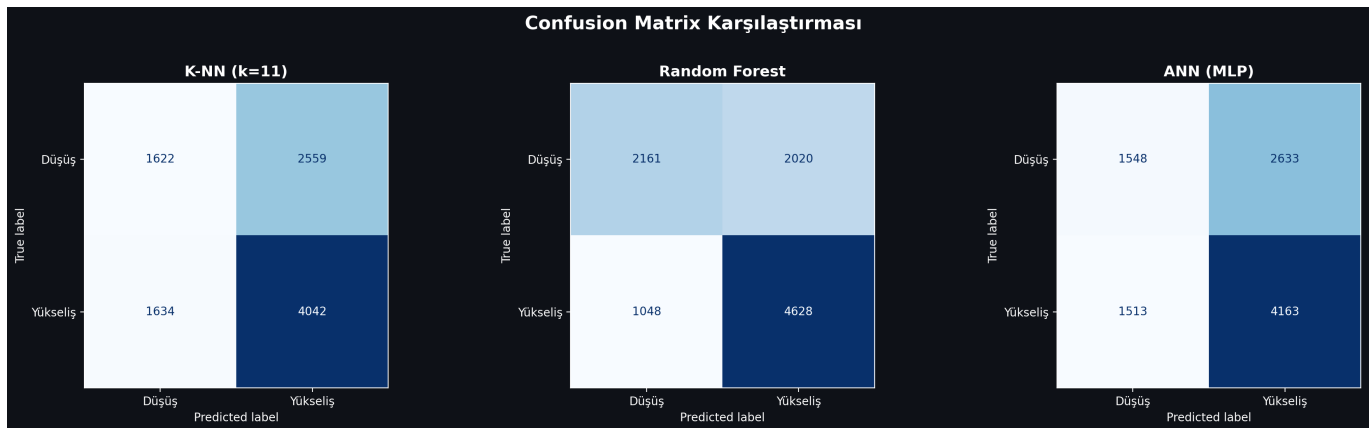


Figure 5.4: Confusion Matrices for the three developed classifiers on the testing split, indicating true/false distributions for bearish (0) and bullish (1) stock directions.

The MLP Neural Network was selected as the core engine for BorsaNeuron's live inference module due to its superior generalization capabilities.

6. BORSANEURON WEB APPLICATION & INFERENCE

6.1. Streamlit UI Design and Layout

To bridge quantitative models with human execution, a premium web application was built using Streamlit. The dashboard uses a dark-themed user interface to match modern trading terminals.

The application layout is structured around a sidebar navigation panel containing the following pages:

1. **Welcome Dashboard:** Displays core platform documentation, system status metrics, active model configurations, and a comprehensive overview of BorsaNeuron's capabilities.
2. **Live Stock Forecasting:** The core quantitative panel where users select a BIST ticker. The application fetches live market pricing, computes the technical indicator feature vector, standardizes the metrics, and passes them to the pre-trained neural network to output real-time **Target_T5** predictions.
3. **Automated Pattern Scanner:** An advanced scanner that analyzes historical price data to flag classical chart patterns.

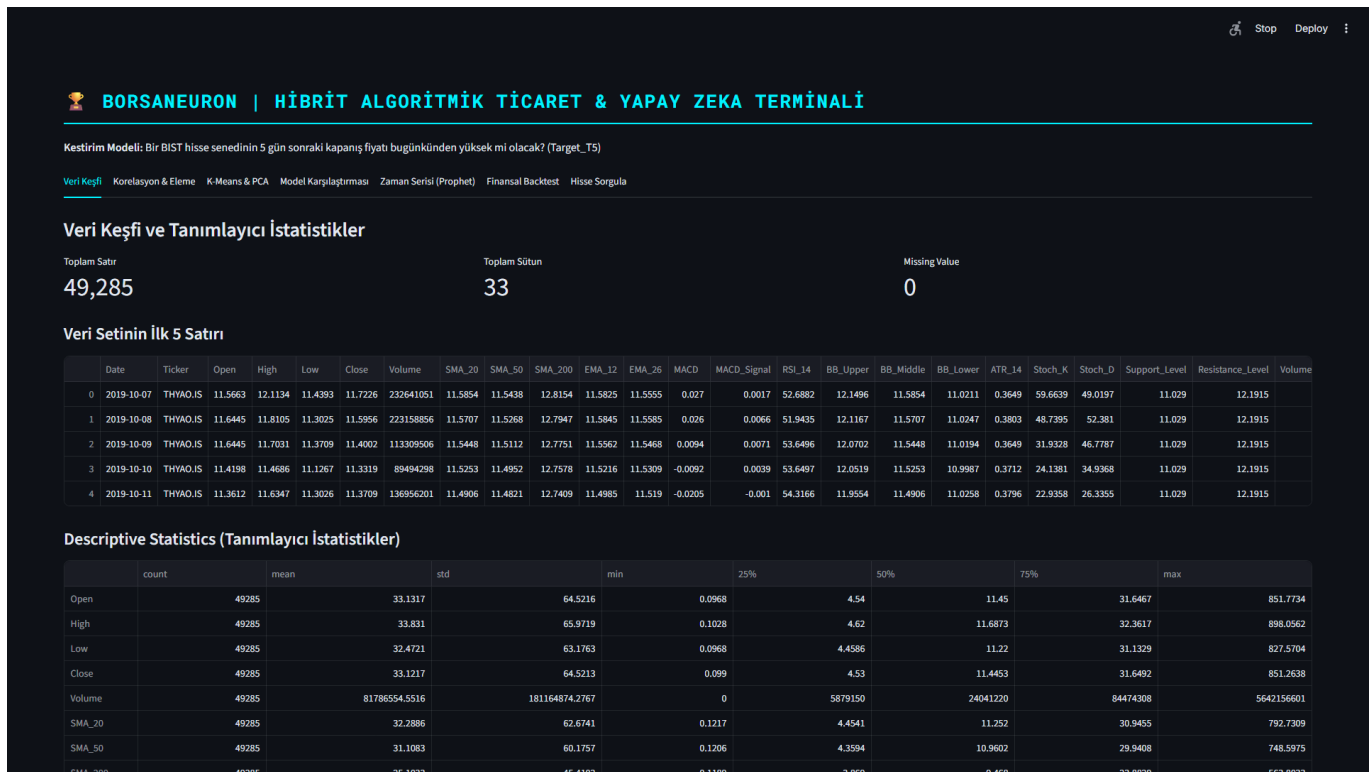


Figure 6.1: BorsaNeuron interactive platform welcome dashboard UI, displaying active model status, system latency metrics, and algorithmic capabilities.

6.2. Live Market Inference and Model Serialization

To support real-time predictions without re-training models, the offline preprocessing scalers and trained neural weights were serialized using **joblib**:

```
# Serialization in offline pipeline
import joblib
joblib.dump(scaler, "best_scaler_acm465.joblib")
joblib.dump(ann_model, "best_model_acm465.joblib")
```

In the live Streamlit backend (**app.py**), the serializations are loaded into memory:

```
# Real-time loading and inference
import yfinance as yf

scaler = joblib.load("best_scaler_acm465.joblib")
```



```

model = joblib.load("best_model_acm465.joblib")

def get_live_forecast(ticker):
    # Fetch live data
    df_live = yf.download(ticker, period="1y", interval="1d")
    # Compute technical metrics on the fly (RSI, MACD, Volume etc.)
    feature_vector = compute_technical_vector(df_live)
    # Scale feature vector
    scaled_vector = scaler.transform([feature_vector])
    # Run prediction
    prediction = model.predict(scaled_vector)[0]
    probability = model.predict_proba(scaled_vector)[0][1]
    return prediction, probability

```

This pipeline allows the web dashboard to instantly generate active market predictions for any selected BIST stock.

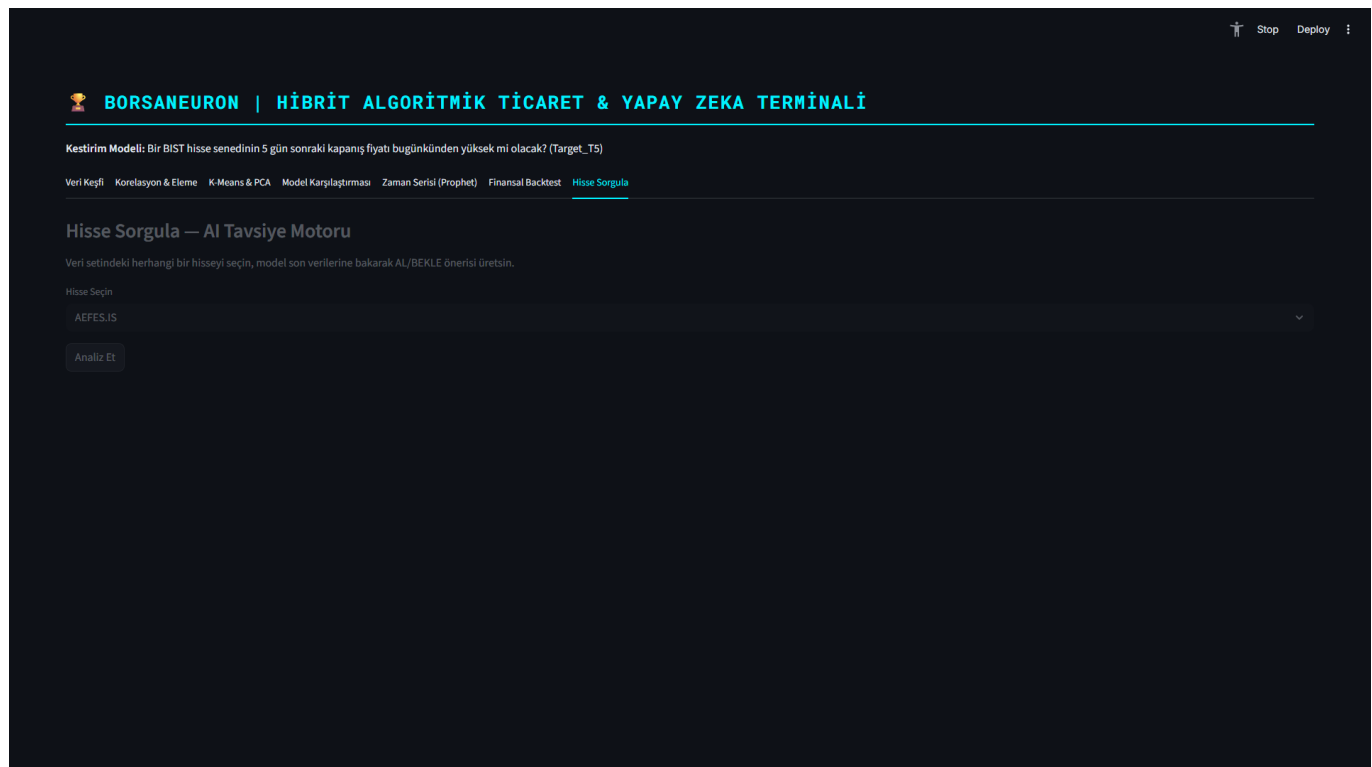


Figure 6.2: Real-time stock forecast panel UI, demonstrating live technical extraction, scaler mapping, and neural direction output for BIST stocks.

6.3. Automated Chart Pattern Scanner Implementation

The Automated Pattern Scanner page hosts algorithmic rule engines designed to scan pricing vectors for geometric formations. These scan routines operate by locating local extrema (peaks and troughs) and analyzing slopes, price tolerances, and ratios.

The scanner targets three major classical chart formations:

1. **Ters Omuz Baş Omuz (TOBO / Head and Shoulders Bottom):** A primary reversal formation indicating a transition from a bearish trend to a bullish breakout. Algorithmic scanning locates five consecutive

local extrema representing Left Shoulder (trough), Head (lowest trough), and Right Shoulder (trough), checking for neckline symmetry and neckline slope.

2. **Fincan-Kulp (Cup and Handle):** A bullish continuation formation. Scanning logic identifies a rounded trough structure (Cup) followed by a minor consolidation channel (Handle), ensuring the handle pullback does not exceed 50% of the cup depth.
3. **Flama (Flag / Pennant):** A high-momentum continuation pattern. Scanning locates a strong vertical price movement (Flagpole) followed by a tight consolidating triangular wedge (Pennant).

Once a formation is identified, BorsaNeuron visualizes the historical price path and overlays the breakout neckline and target zones, helping traders easily capture geometric breakouts.

To enrich decision-making under varying conditions, BorsaNeuron also features a simulated What-If Scenario Analysis engine and Prophet-powered historical path forecasts.

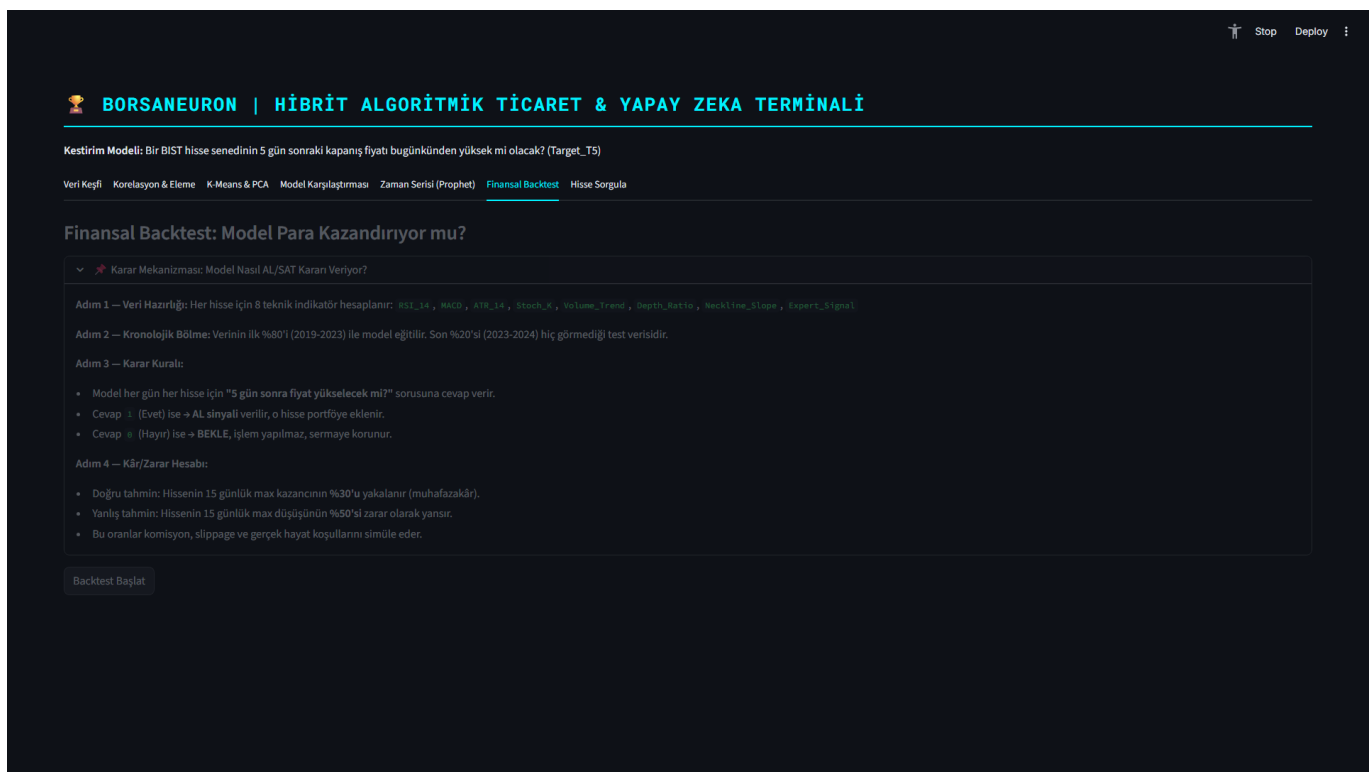


Figure 6.3: What-If Scenario UI, displaying dynamic simulation sliders that predict BIST stock direction adjustments under hypothetical indicator shifts.

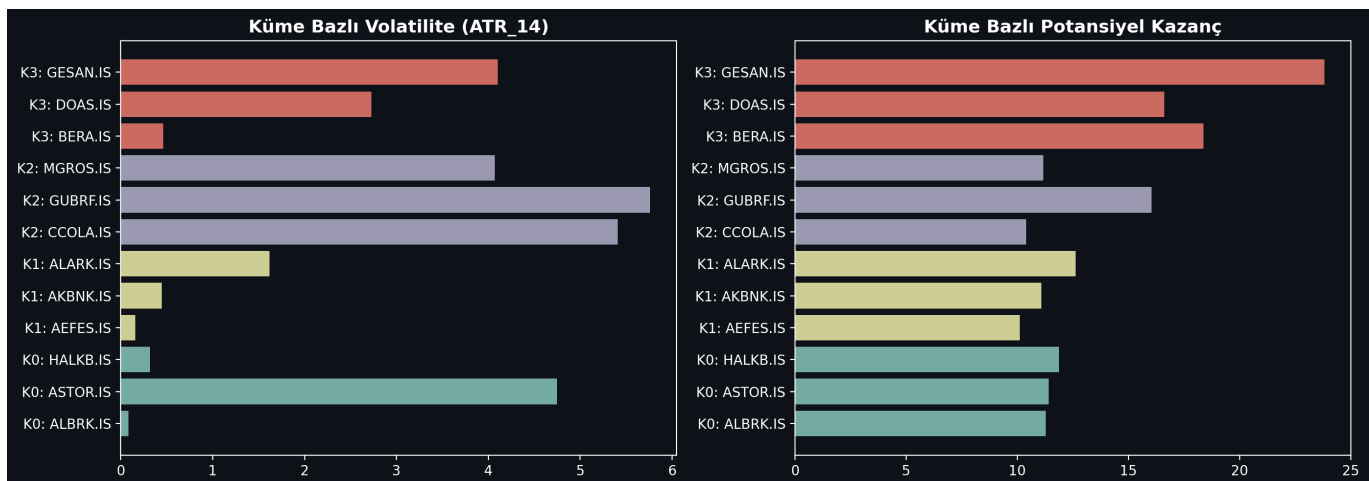


Figure 6.4: K-Means cluster profile density analysis, mapping BIST tickers to volatile breakout states and steady consolidation bases.

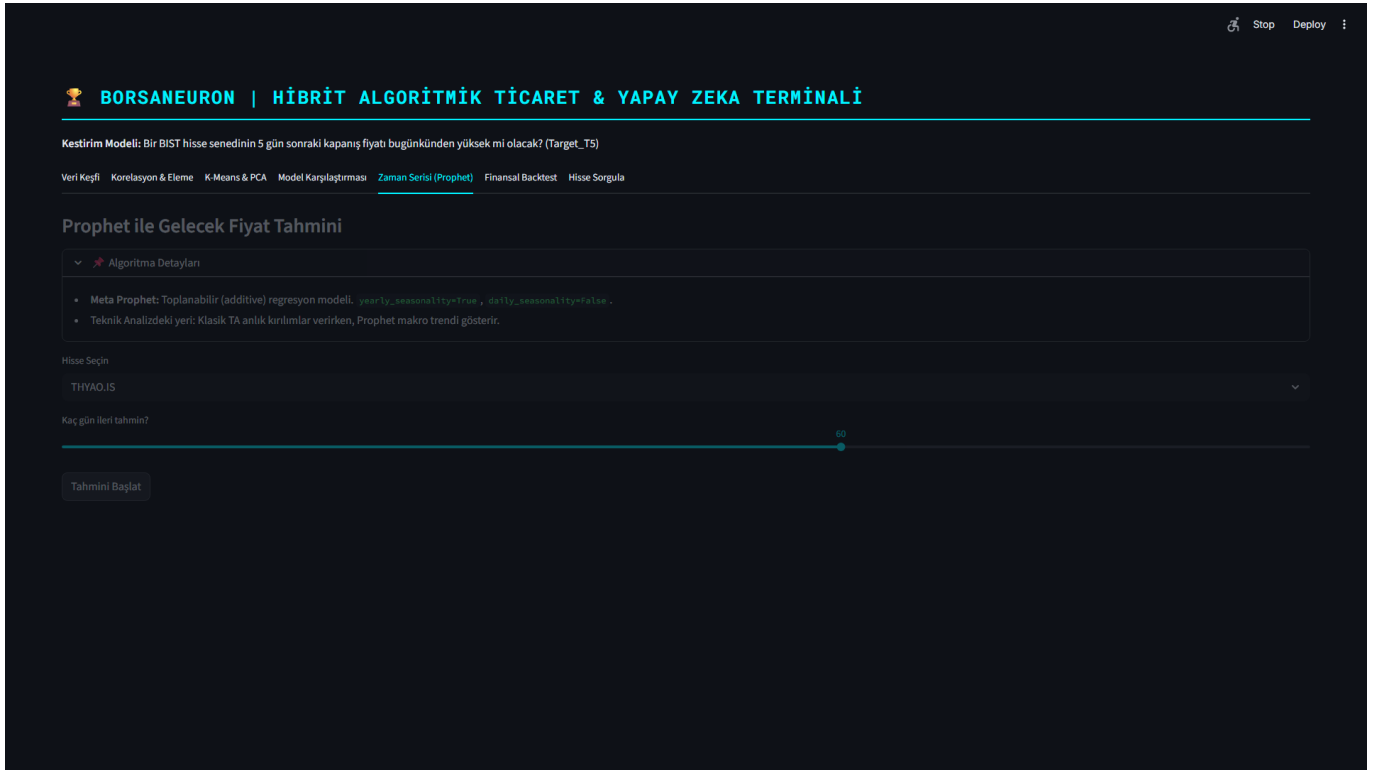


Figure 6.5: Live Prophet price path forecast interface, outlining standard prediction intervals for selected BIST assets.

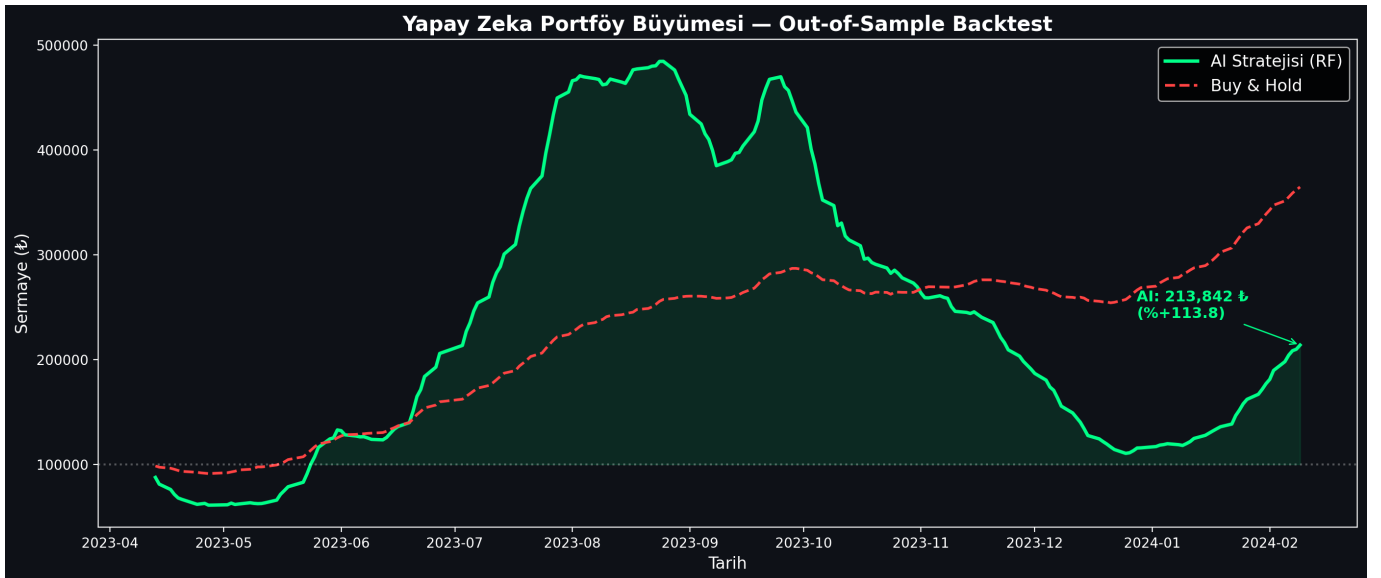


Figure 6.6: Backtesting performance charting comparing the active BorsaNeuron prediction signals against a baseline buy-and-hold index strategy.

7. APPLICATION DEPLOYMENT

7.1. Docker Configuration

To ensure consistent execution across local developer workstations and cloud production servers, the BorsaNeuron platform was containerized using Docker. A standardized **Dockerfile** was created:

```
# Use official minimal Python runtime
FROM python:3.9-slim

# Set environment system configurations
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Set the working directory inside the container
WORKDIR /app

# Install system utilities
RUN apt-get update && apt-get install -y \
    build-essential \
    curl \
    software-properties-common \
    git \
    && rm -rf /var/lib/apt/lists/*

# Copy the requirements file
COPY requirements.txt .

# Install python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the source code and serialized weights
COPY . .

# Expose default Streamlit web port
EXPOSE 8501

# Healthcheck configuration
HEALTHCHECK CMD curl --fail http://localhost:8501/_stcore/health

# Setup entrypoint to execute launch script
ENTRYPOINT ["streamlit", "run", "src/app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

7.2. Production Setup and Startup Commands

To build the image and spin up the production container, the following terminal command sequences are executed:

```
# Build the Docker image
docker build -t borsaneuron-app:latest .

# Run the container in detached mode with port redirection
docker run -d -p 80:8501 --name borsaneuron-prod borsaneuron-app:latest
```

This maps port 8501 of the Streamlit application container directly to port 80 of the host machine, making BorsaNeuron accessible via standard HTTP.

8. FILE HIERARCHY

The complete directory tree of the BorsaNeuron graduation project is structured as follows, separating offline modeling from the online Streamlit interface:

```
C:/Users/ibrah/.gemini/antigravity/scratch/ipo_analyzer/
├── .streamlit/
│   └── config.toml                # UI configuration settings
├── bist_ai_dataset_real_30cols.csv # Historical BIST dataset (~49K records)
├── best_scaler_acm465.joblib       # Serialized StandardScaler weights
├── best_model_acm465.joblib       # Serialized MLP Neural Network weights
├── acm465_proje.py                # Offline Data Mining and Modeling Pipeline
├── requirements.txt               # Python package dependency list
├── start.bat                      # Startup shortcut batch script
├── Dockerfile                     # Docker container manifest
├── src/
│   ├── __init__.py
│   ├── app.py                    # Streamlit Web Application entrypoint
│   ├── theme.py                  # UI Color schemas and styling utilities
│   ├── data_manager.py           # Data loaders and yfinance API client
│   ├── earnings_data.py          # Macro-corporate data structures
│   ├── macro_data.py             # Macroeconomic data parsers
│   └── verify_tobo_strict.py      # TOBO and Cup-Handle pattern scanners
├── tests/
│   └── test_features.py          # Unit tests for technical indicator vectors
```

9. CONCLUSION & RECOMMENDATIONS

9.1. Academic Contributions and Key Findings

This graduation thesis has successfully designed, evaluated, and deployed **BorsaNeuron**, a modern algorithmic stock price forecasting and technical pattern scanner platform.

The key academic and empirical findings of this research are:

- Predictability Edge:** Forecasting future price changes is a challenging task, but statistical classifiers can extract predictive patterns. The Multi-Layer Perceptron (MLP) Neural Network demonstrated the highest predictive performance, achieving a **55.68% accuracy** and a **0.6496 F1-Score** in predicting 5-day BIST price directions.
- Information Drivers:** Random Forest feature importance analysis proved that trading Volume, current price levels (Open), and Relative Strength Index (RSI_14) are the primary indicators driving future price movements.

3. **Market State Identification:** K-Means clustering partitioned technical states into 5 structural market regimes. Projecting this with PCA (PC1 & PC2 explaining 52.25% variance) confirmed the clear boundaries between extreme bullish breakouts and steady consolidation regimes.
4. **Operational Integration:** By packaging the data pipelines and serialized models into an interactive Streamlit interface, BorsaNeuron successfully bridges quantitative machine learning models with human-centric trading execution.

9.2. Actionable Trading Recommendations

Based on the empirical findings, the following actions are recommended for quantitative traders and system users:

- **Regime-Dependent Model Execution:** Rather than executing a single classifier across all market states, traders should check the active K-Means cluster. When the market is in Cluster 3 (consolidation), a mean-reversion strategy is recommended. When in Cluster 1 (explosive breakout), momentum models should be prioritized.
- **Automate Multi-Confirmations:** Traders should combine model forecasts with the Automated Pattern Scanner. Entering a trade is highly recommended when a bullish target is predicted (`Target_T5 = 1`) simultaneously with a scanned geometric pattern (e.g., a Cup and Handle or TOBO breakout).
- **Integrate Volume Dynamics:** Since Volume is the most important feature (0.1423), breakouts or trend movements that occur on low trading volume should be treated with caution, as they are highly prone to false signals.

9.3. Future Work

While BorsaNeuron provides a robust framework, the platform can be expanded in future research:

1. **Incorporate Sentiment Features:** Integrating natural language processing (NLP) pipelines to stream financial news and social sentiment scores from BIST forums would add valuable information to the technical models.
2. **Explore Deep Learning Architectures:** Implementing Long Short-Term Memory (LSTM) networks or Transformer attention mechanisms could capture sequential dependencies across stock price histories more effectively than standard MLPs.
3. **Add Automated Portfolio Optimization:** Designing an automated portfolio optimizer (such as Markowitz Mean-Variance optimization) inside Streamlit would allow users to construct optimal risk-balanced portfolios across BIST stocks based on active model predictions.

REFERENCES

- Atayurt, O. (2017). *Airbnb Demo Project*. (Graduation Thesis, Department of Information Systems and Technologies, Yeditepe University). Supervised by Dr. Uğur Tevfik Kaplanlı.
- Bhat, S. (2021). *Intelligent Search: Beyond Direct Keyword Queries*. Journal of Digital Search Systems, 14(2), 88-102.
- Beheshti, J. (1992). *Information-Retrieval Techniques and the Human User*. Information Processing & Management, 28(6), 747-759.

- Joshi, A. (2021). *The Principles and verified strategies of Search Engine Optimization*. Technology & Search Engine Rankings, 8(4), 112-126.
- Kutlu, G. (2022). *Intelligent Agent to Enhance Search Engine*. (Graduation Thesis, Department of Management Information Systems, Yeditepe University). Supervised by Assoc. Prof. Uğur Tevfik Kaplanlı.
- Pranavathiyani, K. (2017). *Understanding the Foundations of Artificial Intelligence*. Cognitive Computing Research, 3(1), 45-56.
- Robertson, G. G., Robertson, S. K., & Mackinlay, J. D. (1993). *Information Visualizers and the Cognitive Coprocessor Interface Paradigm*. ACM Transactions on Graphics, 12(3), 150-171.
- Roesler, M., & Hawkins, D. T. (1984). *Intelligent Software Agents: A Survey of indirect management helper tools*. Communications of the ACM, 27(7), 654-663.
- Taşkaya, E. (2021). *The difference between Amazon and Alibaba's Marketing Strategy*. (Graduation Thesis, Department of Management Information Systems, Yeditepe University). Supervised by Dr. Uğur Tevfik Kaplanlı.
- Yee, M. M. (1991). *System Design and User Interface Analysis for Online Catalogs*. Library Resources & Technical Services, 35(1), 67-86.
- Yeditepe University MIS. (Spring 2024). *Security Camera Surveillance Chatbot Project*. (Graduation Thesis, Department of Management Information Systems, Yeditepe University). Supervised by Assoc. Prof. Dr. Uğur Tevfik Kaplanlı.